

MFML – Assignment 1

2025AD05069

Gopi Agasthia S

Q1) Finding solutions of linear systems

- a) Write a code taking as input a matrix A of size $m \times n$ and a vector b of size $m \times 1$, where m and n are arbitrarily large numbers and $m < n$, constructing the augmented matrix and performing REF, and RREF without using any built-in functions. In case you encounter any division by 0, you can choose a different A and/or b .

Code for REF:

```
def ref(matrix, verbose=True):
    """Compute the row echelon form of a matrix.

    Parameters:
        matrix (numpy.ndarray): Input matrix.
        verbose (bool): If True, print intermediate steps. Default is True.

    Returns:
        numpy.ndarray: Matrix in row echelon form.
    """
    # Create a working copy as float to allow division
    A = matrix.astype(float).copy()
    rows, cols = A.shape
    pivot_row = 0 # Track which row we're currently working on

    if verbose:
        print("\n=== Row Echelon Form (REF) ===")
        print("\nInitial Matrix:")
        print(A)

    # Process each column to find and use pivots
    for col in range(cols):
        # Step 1: Find a non-zero pivot in the current column
        pivot = None
        for r in range(pivot_row, rows):
            if A[r][col] != 0:
                pivot = r
                break

        # If no pivot found, skip this column
        if pivot is None:
            if verbose:
                print(f"\nColumn {col}: No pivot found, skipping")
            continue

        # Step 2: Swap rows to move pivot to the correct position
        if pivot != pivot_row:
            A[[pivot_row, pivot]] = A[[pivot, pivot_row]]
            if verbose:
```

```

        print(f"\nStep: Swap row {pivot} with row {pivot_row}")
        print(A)

    # Step 3: Eliminate all entries below the pivot
    if verbose:
        print(f"\nEliminating below pivot at position ({pivot_row}, {col})")

    for r in range(pivot_row + 1, rows):
        # Calculate the factor needed to eliminate A[r][col]
        factor = A[r][col] / A[pivot_row][col]

        if verbose and factor != 0:
            print(f"  R{r} = R{r} - ({factor:.3f}) * R{pivot_row}")

        # Subtract factor * pivot_row from current row
        for c in range(col, cols):
            A[r][c] -= factor * A[pivot_row][c]

    if verbose:
        print("Matrix after elimination:")
        print(A)

    # Move to the next pivot row
    pivot_row += 1
    if pivot_row == rows:
        break

# Round to 3 decimal places to avoid floating point errors
result = np.round(A, 3)
if verbose:
    print("\n=== Final REF Matrix ===")
    print(result)

return result

```

RREF:

```

def rref(matrix, verbose=True):

    """Compute the reduced row echelon form of a matrix.

    Parameters:
        matrix (numpy.ndarray): Input matrix.
        verbose (bool): If True, print intermediate steps. Default is True.

    Returns:
        numpy.ndarray: Matrix in reduced row echelon form.
    """

    A = matrix.astype(float).copy()
    rows, cols = A.shape
    pivot_row = 0 # Track which row we're currently working on

    if verbose:
        print("\n=== Reduced Row Echelon Form (RREF) ===")
        print("\nInitial Matrix:")

```

```

print(A)

# Process each column to find and use pivots
for col in range(cols):
    # Step 1: Find a non-zero pivot in the current column
    pivot = None
    for r in range(pivot_row, rows):
        if A[r][col] != 0:
            pivot = r
            break

    # If no pivot found, skip this column
    if pivot is None:
        if verbose:
            print(f"\nColumn {col}: No pivot found, skipping")
        continue

    # Step 2: Swap rows to move pivot to the correct position
    if pivot != pivot_row:
        A[[pivot_row, pivot]] = A[[pivot, pivot_row]]
        if verbose:
            print(f"\nStep: Swap row {pivot} with row {pivot_row}")
        print(A)

    # Step 3: Normalize the pivot row (make pivot = 1)
    pivot_value = A[pivot_row][col]

    if verbose:
        print(f"\nNormalizing row {pivot_row} (dividing by {pivot_value:.3f})")

    # Divide entire row by the pivot value
    for c in range(cols):
        A[pivot_row][c] /= pivot_value

    if verbose:
        print("Matrix after normalization:")
        print(A)

    # Step 4: Eliminate ALL other entries in the pivot column (not just below)
    if verbose:
        print(f"\nEliminating all entries in column {col} except pivot")

    for r in range(rows):
        # Skip the pivot row itself
        if r != pivot_row:
            # Calculate the factor needed to eliminate A[r][col]
            factor = A[r][col]

            if verbose and factor != 0:
                print(f"  R{r} = R{r} - ({factor:.3f}) * R{pivot_row}")

            # Subtract factor * pivot_row from current row
            for c in range(cols):
                A[r][c] -= factor * A[pivot_row][c]

    if verbose:
        print("Matrix after elimination:")

```

```

        print(A)

        # Move to the next pivot row
        pivot_row += 1
        if pivot_row == rows:
            break

# Round to 3 decimal places to avoid floating point errors
result = np.round(A, 3)
if verbose:
    print("\n=== Final RREF Matrix ===")
    print(result)

return result

```

```

def run_q1a(verbose = True):

    # Set seed for reproducibility
    np.random.seed(10)

    # Generate random dimensions
    rows = np.random.randint(1, 6)
    cols = np.random.randint(rows, 6) # Ensure cols >= rows

    # Generate random matrix with values between -10 and 10
    A = np.random.randint(-10, 11, (rows, cols)).astype(float)

    print("\nOriginal Matrix:")
    print(A)

    print("\nREF:")
    print(ref(A, verbose))

    print("\nRREF:")
    print(rref(A, verbose))

```

```

File
(bits-pilani) Gopi ~/Documents/BITS-Pilani/MFML/Assignment 1/Code (main) $ python main.py

===== Q1(a) =====

Original Matrix:
[[ -6.   5. -10.]
 [  7.   6.   7.]]

REF:

=== Row Echelon Form (REF) ===

Initial Matrix:
[[ -6.   5. -10.]
 [  7.   6.   7.]]

Eliminating below pivot at position (0, 0)
  R1 = R1 - (-1.167) * R0
Matrix after elimination:
[[ -6.         5.        -10.         ]
 [  0.         11.83333333 -4.66666667]]

Eliminating below pivot at position (1, 1)
Matrix after elimination:
[[ -6.         5.        -10.         ]
 [  0.         11.83333333 -4.66666667]]

=== Final REF Matrix ===
[[ -6.         5.        -10.         ]
 [  0.         11.833 -4.667]]
[[ -6.         5.        -10.         ]
 [  0.         11.833 -4.667]]

```

RREF:

=== Reduced Row Echelon Form (RREF) ===

Initial Matrix:

```
[[ -6.   5. -10.]  
 [  7.   6.   7.]]
```

Normalizing row 0 (dividing by -6.000)

Matrix after normalization:

```
[[ 1.          -0.83333333  1.66666667]  
 [ 7.           6.           7.         ]]
```

Eliminating all entries in column 0 except pivot

$$R1 = R1 - (7.000) * R0$$

Matrix after elimination:

```
[[ 1.          -0.83333333  1.66666667]  
 [ 0.           11.83333333 -4.66666667]]
```

Normalizing row 1 (dividing by 11.833)

Matrix after normalization:

```
[[ 1.          -0.83333333  1.66666667]  
 [ 0.           1.          -0.3943662 ]]
```

Eliminating all entries in column 1 except pivot

$$R0 = R0 - (-0.833) * R1$$

Matrix after elimination:

```
[[ 1.           0.          1.33802817]  
 [ 0.           1.          -0.3943662 ]]
```

=== Final RREF Matrix ===

```
[[ 1.   0.   1.338]  
 [ 0.   1.  -0.394]]  
[[ 1.   0.   1.338]  
 [ 0.   1.  -0.394]]
```

(bits-pilani) Gopi ~/Documents/BITS-Pilani/MFML/Assignment 1/Code (main) \$

b) Identify the pivot and non-pivot columns and find the particular solution and solutions to $Ax=0$.

```
def get_pivots_and_free_columns(rref_matrix):

    """Identify pivot and free columns from a reduced row echelon matrix.

    Pivot columns correspond to leading variables, while free columns
    correspond to variables that can be chosen freely.

    Parameters:
        rref_matrix (numpy.ndarray): Matrix in reduced row echelon form.

    Returns:
        tuple[list[int], list[int]]: A tuple of pivot columns and free columns.
    """

    rows, cols = rref_matrix.shape
    pivot_columns = []

    for r in range(rows):
        for c in range(cols):
            if abs(rref_matrix[r][c] - 1) < 1e-10:
                is_pivot = True
                for k in range(rows):
                    if k != r and abs(rref_matrix[k][c]) > 1e-10:
                        is_pivot = False
                        break
                if is_pivot:
                    pivot_columns.append(c)
                    break

    free_columns = [c for c in range(cols - 1) if c not in pivot_columns]
    return pivot_columns, free_columns


def particular_solution(rref_matrix, pivot_columns, free_columns, verbose=True):

    """Compute a particular solution for the augmented system in RREF.

    Free variables are set to zero, and pivot variables are assigned the
    constant term from each pivot row.

    Parameters:
        rref_matrix (numpy.ndarray): Augmented matrix in reduced row echelon form.
        pivot_columns (list[int]): Indices of pivot columns.
        free_columns (list[int]): Indices of free variable columns.
        verbose (bool): If True, print intermediate steps. Default is True.

    Returns:
        numpy.ndarray: Particular solution vector.
    """

    rows, cols = rref_matrix.shape
    variables = cols - 1
    x = np.zeros(variables)
```

```

if verbose:
    print("\n=== Computing Particular Solution ===")
    print("\nStrategy: Set all free variables to 0, solve for pivot variables")
    print(f"Free variables (set to 0): {free_columns}")
    print(f"Pivot variables (solve from RREF): {pivot_columns}")
    print("\nInitial solution vector (all zeros):")
    print(x)

for r in range(rows):
    pivot_col = None
    for c in pivot_columns:
        if abs(rref_matrix[r][c] - 1) < 1e-10:
            pivot_col = c
            break
    if pivot_col is not None:
        if verbose:
            print(f"\nRow {r}: Pivot at column {pivot_col}")
            print(f"    Setting x[{pivot_col}] = {rref_matrix[r][-1]:.1f} (constant term)")
        x[pivot_col] = rref_matrix[r][-1]
        if verbose:
            print(f"    Current solution: {x}")

if verbose:
    print("\n=== Final Particular Solution ===")
    print(x)

return x

def null_space_basis(rref_matrix, pivot_columns, free_columns, verbose=True):

    """Construct a basis for the null space of the coefficient matrix.

    Each free variable is set to 1 in turn, and pivot variables are computed
    to satisfy the homogeneous system  $Ax = 0$ .

    Parameters:
        rref_matrix (numpy.ndarray): Augmented matrix in reduced row echelon form.
        pivot_columns (list[int]): Indices of pivot columns.
        free_columns (list[int]): Indices of free variable columns.
        verbose (bool): If True, print intermediate steps. Default is True.

    Returns:
        list[numpy.ndarray]: Basis vectors for the null space.
    """

    rows, cols = rref_matrix.shape
    variables = cols - 1
    basis = []

    if verbose:
        print("\n=== Computing Null Space Basis ===")
        print("\nStrategy: Set each free variable to 1 (one at a time), solve for pivot
variables")
        print(f"Number of free variables: {len(free_columns)}")
        print(f"Free variable columns: {free_columns}")

```



```

    print(f"Pivot variable columns: {pivot_columns}")

    for idx, free_col in enumerate(free_columns):
        if verbose:
            print(f"\n--- Basis Vector {idx + 1} (for free variable x{free_col + 1}) ---")

        vector = np.zeros(variables)
        vector[free_col] = 1

        if verbose:
            print(f"Set x[{free_col}] = 1 (free variable)")
            print(f"Initial vector: {vector}")

        for r in range(rows):
            pivot_col = None
            for c in pivot_columns:
                if abs(rref_matrix[r][c] - 1) < 1e-10:
                    pivot_col = c
                    break
            if pivot_col is not None:
                # Express the pivot variable in terms of the current free variable.
                coeff = -rref_matrix[r][free_col]
                if verbose:
                    print(f"\nRow {r}: Pivot at column {pivot_col}")
                    print(f"  From RREF: x[{pivot_col}] +
({rref_matrix[r][free_col]:.1f}) * x[{free_col}] = 0")
                    print(f"  Therefore: x[{pivot_col}] = {coeff:.1f}")
                vector[pivot_col] = coeff
                if verbose:
                    print(f"  Current vector: {vector}")

        basis.append(vector)

        if verbose:
            print(f"\nBasis vector {idx + 1}: {vector}")

    if verbose:
        print("\n=== Final Null Space Basis ===")
        for idx, vec in enumerate(basis):
            print(f"v{idx + 1} = {vec}")

    return basis

```

```

def run_q1b(verbose=True):
    np.random.seed(15)

    rows = np.random.randint(1, 6)
    cols = np.random.randint(rows, 6) # Ensure cols >= rows for potential null space

    A = np.random.randint(-10, 11, (rows, cols)).astype(float)

    R = rref(A, verbose)

    print("\nRREF:")
    print(R)

```

```

pivot_columns, free_columns = \
    get_pivots_and_free_columns(R)

print("\nPivot Columns:")
print(pivot_columns)

print("\nFree Columns:")
print(free_columns)

particular = particular_solution(
    R,
    pivot_columns,
    free_columns,
    verbose
)

print("\nParticular Solution:")
print(particular)

basis = null_space_basis(
    R,
    pivot_columns,
    free_columns,
    verbose
)

print("\nNull Space Basis:")

for i, vec in enumerate(basis):
    print(f"v{i+1} = {vec}")

# Print general solution
print("\n=== General Solution ===")
print("\nThe general solution is:")
print("x = x_particular + c1*v1 + c2*v2 + ... + ck*v_k")
print("\nFor this system:")
print(f"x_particular = {particular}")

if len(basis) == 0:
    print("\nGeneral solution: x = x_particular (unique solution)")
else:
    print("\nGeneral solution:")
    solution_str = f"x = {particular}"
    for i, vec in enumerate(basis):
        solution_str += f" + c{i+1}*{vec}"
    print(solution_str)

    print("\nIn component form:")
    for j in range(len(particular)):
        component_str = f"x{j+1} = {particular[j]:.1f}"
        for i, vec in enumerate(basis):
            if vec[j] != 0:
                component_str += f" + {vec[j]:.1f}*c{i+1}" if vec[j] > 0 else f" -
{abs(vec[j]):.1f}*c{i+1}"
        print(component_str)

    print(f"\nwhere c1, c2, ..., c{len(basis)} are free parameters (any real numbers)")

```

```
(bits-pilani) Gopi ~/Documents/BITS-Pilani/MFML/Assignment 1/Code (main) $ python main.py
```

```
===== Q1(b) =====
```

```
=== Reduced Row Echelon Form (RREF) ===
```

```
Initial Matrix:
```

```
[[ -5. -10.  -3.   1.   5.]]
```

```
Normalizing row 0 (dividing by -5.000)
```

```
Matrix after normalization:
```

```
[[ 1.   2.   0.6 -0.2 -1. ]]
```

```
Eliminating all entries in column 0 except pivot
```

```
Matrix after elimination:
```

```
[[ 1.   2.   0.6 -0.2 -1. ]]
```

```
=== Final RREF Matrix ===
```

```
[[ 1.   2.   0.6 -0.2 -1. ]]
```

```
RREF:
```

```
[[ 1.   2.   0.6 -0.2 -1. ]]
```

```
Pivot Columns:
```

```
[0]
```

```
Free Columns:
```

```
[1, 2, 3]
```

=== Computing Particular Solution ===

Strategy: Set all free variables to 0, solve for pivot variables

Free variables (set to 0): [1, 2, 3]

Pivot variables (solve from RREF): [0]

Initial solution vector (all zeros):

[0. 0. 0. 0.]

Row 0: Pivot at column 0

Setting $x[0] = -1.0$ (constant term)

Current solution: [-1. 0. 0. 0.]

=== Final Particular Solution ===

[-1. 0. 0. 0.]

Particular Solution:

[-1. 0. 0. 0.]

=== Computing Null Space Basis ===

Strategy: Set each free variable to 1 (one at a time), solve for pivot variables

Number of free variables: 3

Free variable columns: [1, 2, 3]

Pivot variable columns: [0]

--- Basis Vector 1 (for free variable x_2) ---

Set $x[1] = 1$ (free variable)

Initial vector: [0. 1. 0. 0.]

Row 0: Pivot at column 0

From RREF: $x[0] + (2.0) \cdot x[1] = 0$

Therefore: $x[0] = -2.0$

Current vector: [-2. 1. 0. 0.]

Basis vector 1: [-2. 1. 0. 0.]

```
--- Basis Vector 2 (for free variable x3) ---  
Set x[2] = 1 (free variable)  
Initial vector: [0. 0. 1. 0.]
```

```
Row 0: Pivot at column 0  
From RREF:  $x[0] + (0.6)x[2] = 0$   
Therefore:  $x[0] = -0.6$   
Current vector: [-0.6 0. 1. 0.]
```

```
Basis vector 2: [-0.6 0. 1. 0.]
```

```
--- Basis Vector 3 (for free variable x4) ---  
Set x[3] = 1 (free variable)  
Initial vector: [0. 0. 0. 1.]
```

```
Row 0: Pivot at column 0  
From RREF:  $x[0] + (-0.2)x[3] = 0$   
Therefore:  $x[0] = 0.2$   
Current vector: [0.2 0. 0. 1.]
```

```
Basis vector 3: [0.2 0. 0. 1.]
```

```
=== Final Null Space Basis ===
```

```
v1 = [-2. 1. 0. 0.]  
v2 = [-0.6 0. 1. 0.]  
v3 = [0.2 0. 0. 1.]
```

```
Null Space Basis:
```

```
v1 = [-2. 1. 0. 0.]  
v2 = [-0.6 0. 1. 0.]  
v3 = [0.2 0. 0. 1.]
```

```
=== General Solution ===
```

```
The general solution is:
```

```
 $x = x_{\text{particular}} + c_1*v_1 + c_2*v_2 + \dots + c_k*v_k$ 
```

```
For this system:
```

```
 $x_{\text{particular}} = [-1. 0. 0. 0.]$ 
```

```
General solution:
```

```
 $x = [-1. 0. 0. 0.] + c1*[-2. 1. 0. 0.] + c2*[-0.6 0. 1. 0.] + c3*[0.2 0. 0. 1.]$ 
```

- Q. a) Finding Row Echelon Form and Reduced Row echelon Form for a matrix 'A' of size $m \times n$ given $m < n$

Solution

Let's take equations :

$$x_1 + 2x_2 + 3x_3 + 4x_4 = 5$$

$$2x_1 + 4x_2 + 6x_3 + 8x_4 = 10$$

$$x_3 + 2x_4 = 3$$

Matrix $[A | b]$

$$\begin{bmatrix} 1 & 2 & 3 & 4 & | & 5 \\ 2 & 4 & 6 & 8 & | & 10 \\ 0 & 0 & 1 & 2 & | & 3 \end{bmatrix}$$

$R_1 \rightarrow$ Pivot is already 1

$R_2 \rightarrow R_2 = R_2 - 2R_1$

$$\begin{bmatrix} 1 & 2 & 3 & 4 & | & 5 \\ 0 & 0 & 0 & 0 & | & 0 \\ 0 & 0 & 1 & 2 & | & 3 \end{bmatrix}$$

$$R_2 \longleftrightarrow R_3$$

$$\text{REF: } \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 0 & 0 & 1 & 2 & 3 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

This matrix is in Row Echelon Form

For Reduced Row Echelon Form

$$R_1 = R_1 - 3R_2$$

$$\text{RREF: } \left[\begin{array}{cccc|c} 1 & 2 & 0 & -2 & -4 \\ 0 & 0 & 1 & 2 & 3 \\ 0 & 0 & 0 & 0 & 0 \end{array} \right]$$

b) Finding pivot and non pivot columns and particular and general solutions to $Ax = 0$

Sol

From Q1.a,

$$\text{RREF} = \left[\begin{array}{cccc|c} 1 & 2 & 0 & -2 & -4 \\ 0 & 0 & 1 & 2 & 3 \\ 0 & 0 & 0 & 0 & 0 \end{array} \right]$$

Pivot : C_1, C_3 [columns with leading '1's]

non pivot : C_2, C_4 [$x_2, x_4 \rightarrow$ Free vars]

substituting $x_2, x_4 = 0$,

$$x_1 = -4$$

$$x_3 = 3$$

Particular solution : $\begin{bmatrix} -4 & 0 & 3 & 0 \end{bmatrix}$

Null space Basis:

Setting $x_2 = 1$ & $x_4 = 0$,

$$x_1 + 2 = 0$$

$$x_3 = 0$$

$$x_1 = -2$$

$$v_1 = \begin{bmatrix} -2 & 1 & 0 & 0 \end{bmatrix}$$

setting $x_2 = 0$ & $x_4 = 1$,

$$x_1 - 2 = 0$$

$$x_3 + 2 = 0$$

$$x_1 = 2, x_3 = -2$$

$$v_2 = \begin{bmatrix} 2 & 0 & -2 & 1 \end{bmatrix}$$

General solution : $\begin{bmatrix} -4 & 0 & 3 & 0 \end{bmatrix} +$
 $C_1 \begin{bmatrix} -2 & 1 & 0 & 0 \end{bmatrix} +$
 $C_2 \begin{bmatrix} 2 & 0 & -2 & 1 \end{bmatrix}$

- c) Consider a random 6×9 matrix A and a suitable b and show the REF, RREF, pivot columns, non-pivot columns, the particular solution, the solutions to $Ax = 0$, the general solution and verify the general solution.

```
def run_q1c(verbose=True):

    np.random.seed(10)
    A = np.random.randint(-9, 10, (6, 9)).astype(float)
    b = np.random.randint(-9, 10, (6, 1)).astype(float)
    Aug = np.hstack((A, b))

    print("\nMatrix A:")
    print(A)

    print("\nVector b:")
    print(b)

    REF = ref(Aug, verbose)
    print("\nREF:")
    print(REF)

    RREF = rref(Aug, verbose)

    print("\nRREF:")
    print(RREF)
    pivot_columns, free_columns = get_pivots_and_free_columns(RREF)

    print("\nPivot Columns:")
    print(pivot_columns)

    print("\nFree Columns:")
    print(free_columns)

    particular = particular_solution(
        RREF,
        pivot_columns,
        free_columns,
        verbose
    )

    print("\nParticular Solution:")
    print(particular)

    basis = null_space_basis(
        RREF,
        pivot_columns,
        free_columns,
        verbose
    )

    print("\nNull Space Basis:")

    for vec in basis:
        print(vec)
```

```
(bits-pilani) Gopi ~/Documents/BITS-Pilani/MFML/Assignment 1/Code (main) $ python main.py
```

```
===== Q1(c) =====
```

Matrix A:

```
[[ 0. -5.  6. -9.  8.  7.  8. -1.  0.]  
 [-9.  1. -1. -5.  7. -5.  6.  2.  2.]  
 [-8. -1. -5.  5.  8.  4. -4.  4.  4.]  
 [ 3. -8. -5.  9.  4.  2.  1.  0.  6.]  
 [ 9.  7. -2.  2.  8.  5. -2.  2. -8.]  
 [-9.  3. -4. -5. -2.  9.  3. -7. -1.]]
```

Vector b:

```
[[ 6.]  
 [ 4.]  
 [ 6.]  
 [-3.]  
 [ 1.]  
 [ 2.]]
```

REF:

```
[[ -9.    1.    -1.    -5.    7.    -5.    6.    2.    2.  
  4.    ]  
 [  0.   -5.    6.   -9.    8.    7.    8.   -1.    0.  
  6.    ]  
 [  0.    0.   -6.378 12.844 -1.244  5.8  -12.356  2.6   2.222  
  0.178]  
 [  0.    0.    0.   -8.136 -3.098 -23.617 18.889 -3.725  1.603  
 -11.272]  
 [  0.    0.   -0.    0.   28.076 29.127 -5.523  6.971 -4.51  
 20.475]  
 [  0.    0.    0.    0.   -0.   34.208 -10.559 -6.487 -4.775  
  9.854]]
```

RREF:

```
[[ 1.    0.    0.    0.    0.    0.    0.48 -0.31 -0.344 -0.552]  
 [ 0.    1.    0.    0.    0.    0.   -0.784  0.364 -0.655  0.31 ]  
 [-0.   -0.    1.    0.    0.    0.   -1.333  1.022 -0.041  0.926]  
 [-0.   -0.   -0.    1.    0.    0.   -1.473  0.839  0.214  0.385]  
 [ 0.    0.    0.    0.    1.    0.    0.123  0.445 -0.016  0.43 ]  
 [ 0.    0.    0.    0.    0.    1.   -0.309 -0.19 -0.14  0.288]]
```

Pivot Columns:

```
[0, 1, 2, 3, 4, 5]
```

Free Columns:

```
[6, 7, 8]
```

Particular Solution:

```
[-0.552  0.31  0.926  0.385  0.43  0.288  0.    0.    0.    ]
```

Null Space Basis:

```
[-0.48  0.784  1.333  1.473 -0.123  0.309  1.    0.    0.    ]  
 [ 0.31 -0.364 -1.022 -0.839 -0.445  0.19  0.    1.    0.    ]  
 [ 0.344  0.655  0.041 -0.214  0.016  0.14  0.    0.    1.    ]
```

```
(bits-pilani) Gopi ~/Documents/BITS-Pilani/MFML/Assignment 1/Code (main) $
```

Q2) Matrix decompositions

- a) Consider a symmetric and positive definite matrix A of size $n \times n$. Write a code to construct an elementary matrix for every elementary row operation that is performed on A and using the theory explained in the class write the decomposition of A as LU , where L is a lower triangular matrix and U is an upper triangular matrix.

Code for LU Decomposition:

```
def lu_decomposition(A, verbose=True):
    """
    Perform LU decomposition on a square matrix.

    Decomposes matrix A into:
    - L: Lower triangular matrix with 1s on diagonal
    - U: Upper triangular matrix
    - Elementary matrices: The sequence of elimination matrices used

    Such that  $A = L * U$ 

    Parameters:
        A (numpy.ndarray): Square matrix to decompose
        verbose (bool): If True, print intermediate steps. Default is True.

    Returns:
        tuple: (L, U, elementary_matrices)
            - L: Lower triangular matrix
            - U: Upper triangular matrix
            - elementary_matrices: List of elementary matrices used
    """
    # Convert to float for division operations
    A = A.astype(float)
    n = A.shape[0]

    # Initialize U as a copy of A
    U = A.copy()

    # Initialize L as identity matrix
    L = np.eye(n)

    # Track all elementary matrices used in the process
    elementary_matrices = []

    if verbose:
        print("\n=== LU Decomposition ===")
        print("\nInitial Matrix A:")
        print(A)
        print("\nInitial L (identity):")
        print(L)
        print("\nInitial U (copy of A):")
        print(U)

    # Process each column
    for col in range(n - 1):
```

```

pivot = U[col][col]

if verbose:
    print(f"\n--- Processing column {col} with pivot = {pivot:.3f} ---")

# Eliminate all entries below the pivot
for row in range(col + 1, n):
    # Calculate the multiplier (factor) needed to eliminate U[row][col]
    factor = U[row][col] / pivot

    # Store the factor in L matrix
    L[row][col] = factor

    # Create elementary matrix for this elimination step
    E = np.eye(n)
    E[row][col] = -factor
    elementary_matrices.append(E)

    if verbose:
        print(f"\nEliminating U[{row}][{col}]:")
        print(f"  Factor = U[{row}][{col}] / U[{col}][{col}] = {U[row][col]:.3f} / {pivot:.3f} = {factor:.3f}")
        print(f"  L[{row}][{col}] = {factor:.3f}")
        print(f"  R{row} = R{row} - ({factor:.3f}) * R{col}")

    # Perform row operation: row = row - factor * pivot_row
    for k in range(n):
        U[row][k] -= factor * U[col][k]

    if verbose:
        print(f"\nElementary Matrix E{len(elementary_matrices)}:")
        print(np.round(E, 3))
        print("\nCurrent U:")
        print(np.round(U, 3))
        print("\nCurrent L:")
        print(np.round(L, 3))

if verbose:
    print("\n=== Final LU Decomposition ===")
    print("\nL (Lower triangular):")
    print(np.round(L, 3))
    print("\nU (Upper triangular):")
    print(np.round(U, 3))

return L, U, elementary_matrices

def run_q2a(verbose=True):

    # Set seed for reproducibility
    np.random.seed(20)

    # Generate random dimension [1-5] for square matrix (LU requires square)
    n = np.random.randint(1, 6)

    # Generate random matrix and create symmetric positive definite matrix
    M = np.random.randint(1, 10, (n, n)).astype(float)

```

```
A = np.dot(M.T, M)

print("\nMatrix A:")
print(A)

L, U, elementary_matrices = \
    lu_decomposition(A, verbose)

print("\nL:")
print(L)

print("\nU:")
print(U)

print("\nElementary Matrices:")

for i, E in enumerate(elementary_matrices):
    print(f"\nE{i+1}")
    print(np.round(E, 3))

print("\nVerification:")
print(np.allclose(A, np.dot(L, U)))
```

```
(bits-pilani) Gopi ~/Documents/BITS-Pilani/MFML/Assignment 1/Code (main) $ python main.py
```

```
===== Q2(a) =====
```

```
Matrix A:
```

```
[[ 43.  52.  85.  55.]  
 [ 52. 135. 174. 106.]  
 [ 85. 174. 258. 175.]  
 [ 55. 106. 175. 130.]]
```

```
=== LU Decomposition ===
```

```
Initial Matrix A:
```

```
[[ 43.  52.  85.  55.]  
 [ 52. 135. 174. 106.]  
 [ 85. 174. 258. 175.]  
 [ 55. 106. 175. 130.]]
```

```
Initial L (identity):
```

```
[[1. 0. 0. 0.]  
 [0. 1. 0. 0.]  
 [0. 0. 1. 0.]  
 [0. 0. 0. 1.]]
```

```
Initial U (copy of A):
```

```
[[ 43.  52.  85.  55.]  
 [ 52. 135. 174. 106.]  
 [ 85. 174. 258. 175.]  
 [ 55. 106. 175. 130.]]
```

```
--- Processing column 0 with pivot = 43.000 ---
```

```
Eliminating U[1][0]:
```

```
Factor = U[1][0] / U[0][0] = 52.000 / 43.000 = 1.209
```

```
L[1][0] = 1.209
```

```
R1 = R1 - (1.209) * R0
```

```
Elementary Matrix E1:
```

```
[[ 1.  0.  0.  0. ]  
 [-1.209 1.  0.  0. ]  
 [ 0.  0.  1.  0. ]  
 [ 0.  0.  0.  1. ]]
```

```
Current U:
```

```
[[ 43.  52.  85.  55. ]  
 [ 0.  72.116  71.209  39.488]  
 [ 85.  174.  258.  175. ]  
 [ 55.  106.  175.  130. ]]
```

--- Processing column 2 with pivot = 19.663 ---

Eliminating U[3][2]:

$$\text{Factor} = U[3][2] / U[2][2] = 27.287 / 19.663 = 1.388$$

$$L[3][2] = 1.388$$

$$R3 = R3 - (1.388) * R2$$

Elementary Matrix E6:

```
[[ 1.    0.    0.    0. ]
 [ 0.    1.    0.    0. ]
 [ 0.    0.    1.    0. ]
 [ 0.    0.   -1.388  1. ]]
```

Current U:

```
[[43.    52.    85.    55. ]
 [ 0.    72.116 71.209 39.488]
 [ 0.     0.    19.663 27.287]
 [ 0.     0.     0.     0.161]]
```

Current L:

```
[[1.    0.    0.    0. ]
 [1.209 1.    0.    0. ]
 [1.977 0.987 1.    0. ]
 [1.279 0.548 1.388 1. ]]
```

=== Final LU Decomposition ===

L (Lower triangular):

```
[[1.    0.    0.    0. ]
 [1.209 1.    0.    0. ]
 [1.977 0.987 1.    0. ]
 [1.279 0.548 1.388 1. ]]
```

U (Upper triangular):

```
[[43.    52.    85.    55. ]
 [ 0.    72.116 71.209 39.488]
 [ 0.     0.    19.663 27.287]
 [ 0.     0.     0.     0.161]]
```


b) For the above problem, work out the Cholesky's decomposition.

```
def cholesky_decomposition(A, verbose=True):
    """
    Perform Cholesky decomposition on a symmetric positive definite matrix.

    Decomposes matrix A into:
    - L: Lower triangular matrix

    Such that  $A = L * L^T$ 

    The algorithm computes L column by column:
    - Diagonal elements:  $L[j][j] = \sqrt{A[j][j] - \sum(L[j][k]^2 \text{ for } k < j)}$ 
    - Off-diagonal elements:  $L[i][j] = (A[i][j] - \sum(L[i][k]*L[j][k] \text{ for } k < j)) / L[j][j]$ 

    Parameters:
        A (numpy.ndarray): Symmetric positive definite matrix to decompose
        verbose (bool): If True, print intermediate steps. Default is True.

    Returns:
        numpy.ndarray: Lower triangular matrix L
    """
    n = A.shape[0]

    # Initialize L as zero matrix
    L = np.zeros((n, n))

    if verbose:
        print("\n=== Cholesky Decomposition ===")
        print("\nInitial Matrix A (must be symmetric positive definite):")
        print(A)
        print("\nInitial L (zeros):")
        print(L)

    # Process each row
    for i in range(n):
        for j in range(i + 1):
            if i == j:
                # Sum of squares of all previous elements in this row
                sum_value = 0
                for k in range(j):
                    sum_value += L[j][k] ** 2

                if verbose:
                    print(f"\n--- Computing diagonal element L[{j}][{j}] ---")
                    print(f" Sum of L[{j}][k]^2 for k=0 to {j-1}: {sum_value:.6f}")
                    print(f" L[{j}][{j}] = sqrt(A[{j}][{j}] - sum) = sqrt({A[j][j]:.6f} - {sum_value:.6f})")

                # Diagonal formula:  $\sqrt{A[j][j] - \text{sum of } L[j][k]^2}$ 
                L[j][j] = math.sqrt(A[j][j] - sum_value)

                if verbose:
                    print(f" L[{j}][{j}] = {L[j][j]:.6f}")
                    print("\nCurrent L:")
                    print(np.round(L, 3))
```

```

        else:
            # Sum of products of corresponding elements in rows i and j
            sum_value = 0
            for k in range(j):
                sum_value += L[i][k] * L[j][k]

            if verbose:
                print(f"\n--- Computing off-diagonal element L[{i}][{j}] ---")
                print(f" Sum of L[{i}][k] * L[{j}][k] for k=0 to {j-1}: {sum_value:.6f}")
                print(f" L[{i}][{j}] = (A[{i}][{j}] - sum) / L[{j}][{j}]")
                print(f" L[{i}][{j}] = ({A[i][j]:.6f} - {sum_value:.6f}) / {L[j][j]:.6f}")

            # Off-diagonal formula: (A[i][j] - sum) / L[j][j]
            L[i][j] = (A[i][j] - sum_value) / L[j][j]

            if verbose:
                print(f" L[{i}][{j}] = {L[i][j]:.6f}")
                print("\nCurrent L:")
                print(np.round(L, 3))

# Round to 3 decimal places to avoid floating point errors
L = np.round(L, 3)

if verbose:
    print("\n=== Final Cholesky Decomposition ===")
    print("\nL (Lower triangular):")
    print(L)
    print("\nVerification: L * L^T should equal A")

return L

def run_q2b(verbose=True):

    # Set seed for reproducibility
    np.random.seed(25)

    # Generate random dimension [1-5] for square matrix (Cholesky requires square)
    n = np.random.randint(1, 6)

    # Generate random matrix and create symmetric positive definite matrix
    M = np.random.randint(1, 10, (n, n)).astype(float)
    A = np.dot(M.T, M)

    print("\nMatrix A:")
    print(A)

    L = cholesky_decomposition(A, verbose)

    print("\nL:")
    print(np.round(L, 3))

    print("\nVerification:")
    print(np.allclose(A, np.dot(L, L.T)))

```

```
(bits-pilani) Gopi ~/Documents/BITS-Pilani/MFML/Assignment 1/Code (main) $ python main.py
```

```
===== Q2(b) =====
```

```
Matrix A:
```

```
[[195. 202. 75. 162. 139.]  
 [202. 238. 80. 177. 152.]  
 [ 75.  80. 31.  66.  49.]  
 [162. 177. 66. 175. 114.]  
 [139. 152. 49. 114. 115.]]
```

```
=== Cholesky Decomposition ===
```

```
Initial Matrix A (must be symmetric positive definite):
```

```
[[195. 202. 75. 162. 139.]  
 [202. 238. 80. 177. 152.]  
 [ 75.  80. 31.  66.  49.]  
 [162. 177. 66. 175. 114.]  
 [139. 152. 49. 114. 115.]]
```

```
Initial L (zeros):
```

```
[[0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]]
```

```
--- Computing diagonal element L[0][0] ---
```

```
Sum of L[0][k]^2 for k=0 to -1: 0.000000
```

```
L[0][0] = sqrt(A[0][0] - sum) = sqrt(195.000000 - 0.000000)
```

```
L[0][0] = 13.964240
```

```
Current L:
```

```
[[13.964  0.      0.      0.      0.      ]  
 [ 0.      0.      0.      0.      0.      ]  
 [ 0.      0.      0.      0.      0.      ]  
 [ 0.      0.      0.      0.      0.      ]  
 [ 0.      0.      0.      0.      0.      ]]
```

```
--- Computing off-diagonal element L[1][0] ---
```

```
Sum of L[1][k] * L[0][k] for k=0 to -1: 0.000000
```

```
L[1][0] = (A[1][0] - sum) / L[0][0]
```

```
L[1][0] = (202.000000 - 0.000000) / 13.964240
```

```
L[1][0] = 14.465520
```

```
Current L:
```

--- Computing off-diagonal element L[4][3] ---

Sum of L[4][k] * L[3][k] for k=0 to 2: 110.373686

$L[4][3] = (A[4][3] - \text{sum}) / L[3][3]$

$L[4][3] = (114.000000 - 110.373686) / 5.748505$

$L[4][3] = 0.630827$

Current L:

```
[13.964  0.      0.      0.      0.    ]
[14.466  5.362  0.      0.      0.    ]
[ 5.371  0.43   1.403  0.      0.    ]
[11.601  1.713  2.106  5.749  0.    ]
[ 9.954  1.494 -3.638  0.631  0.    ]]
```

--- Computing diagonal element L[4][4] ---

Sum of L[4][k]^2 for k=0 to 3: 114.947780

$L[4][4] = \sqrt{A[4][4] - \text{sum}} = \sqrt{115.000000 - 114.947780}$

$L[4][4] = 0.228517$

Current L:

```
[13.964  0.      0.      0.      0.    ]
[14.466  5.362  0.      0.      0.    ]
[ 5.371  0.43   1.403  0.      0.    ]
[11.601  1.713  2.106  5.749  0.    ]
[ 9.954  1.494 -3.638  0.631  0.229]]]
```

=== Final Cholesky Decomposition ===

L (Lower triangular):

```
[13.964  0.      0.      0.      0.    ]
[14.466  5.362  0.      0.      0.    ]
[ 5.371  0.43   1.403  0.      0.    ]
[11.601  1.713  2.106  5.749  0.    ]
[ 9.954  1.494 -3.638  0.631  0.229]]]
```

Verification: $L * L^T$ should equal A

L:

```
[13.964  0.      0.      0.      0.    ]
[14.466  5.362  0.      0.      0.    ]
[ 5.371  0.43   1.403  0.      0.    ]
[11.601  1.713  2.106  5.749  0.    ]
[ 9.954  1.494 -3.638  0.631  0.229]]]
```

c) Given n linearly independent vectors in m dimensions (the corresponding matrix with these as columns is A of size $m \times n$), with $m > n$, read about the QR decomposition of a matrix and generate Q and R .

```
def qr_decomposition(A, verbose=True):
    """
    Perform QR decomposition using the Gram-Schmidt orthogonalization process.

    Decomposes matrix A into:
    - Q: Matrix with orthonormal columns ( $Q^T * Q = I$ )
    - R: Upper triangular matrix

    Such that  $A = Q * R$ 

    The Gram-Schmidt process:
    1. For each column of A, start with that column vector
    2. Subtract projections onto all previous orthonormal vectors
    3. Normalize the resulting vector to get the next column of Q
    4. Store the projection coefficients and norm in R

    Parameters:
        A (numpy.ndarray): Matrix to decompose ( $m \times n$ )
        verbose (bool): If True, print intermediate steps. Default is True.

    Returns:
        tuple: (Q, R)
            - Q: Matrix with orthonormal columns ( $m \times n$ )
            - R: Upper triangular matrix ( $n \times n$ )
    """
    m, n = A.shape

    # Initialize Q and R as zero matrices
    Q = np.zeros((m, n))
    R = np.zeros((n, n))

    if verbose:
        print("\n=== QR Decomposition (Gram-Schmidt Process) ===")
        print("\nInitial Matrix A:")
        print(A)
        print(f"\nMatrix dimensions: {m} x {n}")
        print("\nInitial Q (zeros):")
        print(Q)
        print("\nInitial R (zeros):")
        print(R)

    # Process each column of A
    for j in range(n):
        if verbose:
            print(f"\n--- Processing column {j} ---")
            print(f"Starting with v = A[:, {j}]:")
            print(A[:, j])

        # Start with the j-th column of A
        v = A[:, j].copy()

        # Orthogonalize against all previous columns of Q
```

```

for i in range(j):
    # Compute projection coefficient:  $R[i][j] = Q[:, i] \cdot A[:, j]$ 
    R[i][j] = np.dot(Q[:, i], A[:, j])

    if verbose:
        print(f"\nOrthogonalizing against Q[:, {i}]:")
        print(f"  R[{i}][{j}] = Q[:, {i}] \cdot A[:, {j}] = {R[i][j]:.6f}")
        print(f"  v = v - {R[i][j]:.6f} * Q[:, {i}]")

    # Subtract the projection:  $v = v - R[i][j] * Q[:, i]$ 
    v = v - R[i][j] * Q[:, i]

    if verbose:
        print(f"  Updated v:")
        print(f"  {v}")

    # Compute the norm of the orthogonalized vector
    R[j][j] = np.sqrt(np.dot(v, v))

    if verbose:
        print(f"\nNormalizing:")
        print(f"  R[{j}][{j}] = ||v|| = sqrt(v \cdot v) = {R[j][j]:.6f}")
        print(f"  Q[:, {j}] = v / R[{j}][{j}]")

    # Normalize to get the j-th column of Q
    Q[:, j] = v / R[j][j]

    if verbose:
        print(f"\nQ[:, {j}] (normalized):")
        print(np.round(Q[:, j], 3))
        print("\nCurrent Q:")
        print(np.round(Q, 3))
        print("\nCurrent R:")
        print(np.round(R, 3))

# Round to 3 decimal places to avoid floating point errors
Q = np.round(Q, 3)
R = np.round(R, 3)

if verbose:
    print("\n=== Final QR Decomposition ===")
    print("\nQ (Orthonormal columns):")
    print(Q)
    print("\nR (Upper triangular):")
    print(R)
    print("\nVerification: Q * R should equal A")

return Q, R

```

```

def run_q2c(verbose=True):

```

```

    # Set seed for reproducibility
    np.random.seed(30)

```

```

    # Generate random dimensions [1-5] x [1-5] with rows >= cols (tall or square)
    cols = np.random.randint(1, 6)

```

```
rows = np.random.randint(cols, 6) # Ensure rows >= cols for QR

# Generate random matrix with values between 1 and 10
A = np.random.randint(1, 10, (rows, cols)).astype(float)

print("\nMatrix A:")
print(A)

Q, R = qr_decomposition(A, verbose)

print("\nQ:")
print(np.round(Q, 3))

print("\nR:")
print(np.round(R, 3))

print("\nVerification:")
print(np.allclose(A, np.dot(Q, R)))
```

```
(bits-pilani) Gopi ~/Documents/BITS-Pilani/MFML/Assignment 1/Code (main) $ python main.py
```

```
===== Q2(c) =====
```

```
Matrix A:
```

```
[[8. 3. 6. 2. 4.]  
 [8. 8. 2. 2. 4.]  
 [3. 3. 5. 5. 7.]  
 [1. 7. 1. 8. 5.]  
 [3. 2. 8. 2. 9.]]
```

```
=== QR Decomposition (Gram-Schmidt Process) ===
```

```
Initial Matrix A:
```

```
[[8. 3. 6. 2. 4.]  
 [8. 8. 2. 2. 4.]  
 [3. 3. 5. 5. 7.]  
 [1. 7. 1. 8. 5.]  
 [3. 2. 8. 2. 9.]]
```

```
Matrix dimensions: 5 x 5
```

```
Initial Q (zeros):
```

```
[[0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]]
```

```
Initial R (zeros):
```

```
[[0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]]
```

```
--- Processing column 0 ---
```

```
Starting with v = A[:, 0]:
```

```
[8. 8. 3. 1. 3.]
```

```
Normalizing:
```

```
R[0][0] = ||v|| = sqrt(v · v) = 12.124356
```

```
Q[:, 0] = v / R[0][0]
```

```
Q[:, 0] (normalized):
```

```
[0.66 0.66 0.247 0.082 0.247]
```


Normalizing:

$R[0][0] = ||v|| = \text{sqrt}(v \cdot v) = 12.124356$

$Q[:, 0] = v / R[0][0]$

$Q[:, 0]$ (normalized):

[0.66 0.66 0.247 0.082 0.247]

Current Q:

```
[[0.66 0. 0. 0. 0. ]
 [0.66 0. 0. 0. 0. ]
 [0.247 0. 0. 0. 0. ]
 [0.082 0. 0. 0. 0. ]
 [0.247 0. 0. 0. 0. ]]
```

Current R:

```
[[12.124 0. 0. 0. 0. ]
 [ 0. 0. 0. 0. 0. ]
 [ 0. 0. 0. 0. 0. ]
 [ 0. 0. 0. 0. 0. ]
 [ 0. 0. 0. 0. 0. ]]
```

--- Processing column 1 ---

Starting with $v = A[:, 1]$:

[3. 8. 3. 7. 2.]

Orthogonalizing against $Q[:, 0]$:

$R[0][1] = Q[:, 0] \cdot A[:, 1] = 9.072647$

$v = v - 9.072647 * Q[:, 0]$

Updated v:

[-2.98639456 2.01360544 0.75510204 6.25170068 -0.24489796]

Normalizing:

$R[1][1] = ||v|| = \text{sqrt}(v \cdot v) = 7.258586$

$Q[:, 1] = v / R[1][1]$

$Q[:, 1]$ (normalized):

[-0.411 0.277 0.104 0.861 -0.034]

Current Q:

```
[[ 0.66 -0.411 0. 0. 0. ]
 [ 0.66 0.277 0. 0. 0. ]
 [ 0.247 0.104 0. 0. 0. ]
 [ 0.082 0.861 0. 0. 0. ]
 [ 0.247 -0.034 0. 0. 0. ]]
```

Current R:

```
[[12.124 9.073 0. 0. 0. ]
 [ 0. 7.259 0. 0. 0. ]
 [ 0. 0. 0. 0. 0. ]
 [ 0. 0. 0. 0. 0. ]
 [ 0. 0. 0. 0. 0. ]]
```

--- Processing column 4 ---

Starting with $v = A[:, 4]$:

[4. 4. 7. 5. 9.]

Orthogonalizing against $Q[:, 0]$:

$R[0][4] = Q[:, 0] \cdot A[:, 4] = 9.649997$

$v = v - 9.649997 * Q[:, 0]$

Updated v :

[-2.36734694 -2.36734694 4.6122449 4.20408163 6.6122449]

Orthogonalizing against $Q[:, 1]$:

$R[1][4] = Q[:, 1] \cdot A[:, 4] = 4.194891$

$v = v - 4.194891 * Q[:, 1]$

Updated v :

[-0.64144609 -3.53105229 4.17585539 0.59109103 6.75377663]

Orthogonalizing against $Q[:, 2]$:

$R[2][4] = Q[:, 2] \cdot A[:, 4] = 8.648320$

$v = v - 8.648320 * Q[:, 2]$

Updated v :

[-0.65310809 0.4492757 0.74707987 -0.54774746 -0.02094435]

Orthogonalizing against $Q[:, 3]$:

$R[3][4] = Q[:, 3] \cdot A[:, 4] = -0.315695$

$v = v - -0.315695 * Q[:, 3]$

Updated v :

[-0.50889414 0.30502681 0.88875832 -0.45624992 -0.19302881]

Normalizing:

$R[4][4] = ||v|| = \text{sqrt}(v \cdot v) = 1.177850$

$Q[:, 4] = v / R[4][4]$

$Q[:, 4]$ (normalized):

[-0.432 0.259 0.755 -0.387 -0.164]

Current Q :

```
[ [ 0.66 -0.411 0.001 0.457 -0.432]
  [ 0.66 0.277 -0.46 -0.457 0.259]
  [ 0.247 0.104 0.396 0.449 0.755]
  [ 0.082 0.861 0.132 0.29 -0.387]
  [ 0.247 -0.034 0.783 -0.545 -0.164]]
```

Current R :

```
[ [12.124 9.073 8.578 5.031 9.65 ]
  [ 0. 7.259 -0.802 7.075 4.195]
  [ 0. 0. 7.468 3.685 8.648]
  [ 0. 0. 0. 3.472 -0.316]
  [ 0. 0. 0. 0. 1.178]]
```

Q2

Matrix decompositions.

a) LU Decomposition

 $A \rightarrow$ matrix of dim $n \times n$.

Let's take 'A' :

$$\begin{bmatrix} 4 & 3 & 2 \\ 6 & 5 & 4 \\ 2 & 1 & 3 \end{bmatrix}$$

Initial state

$$L \leftrightarrow I \quad ; \quad U \leftrightarrow A$$

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad U = \begin{bmatrix} 4 & 3 & 2 \\ 6 & 5 & 4 \\ 2 & 1 & 3 \end{bmatrix}$$

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 1.5 & 1 & 0 \\ 0.5 & 0 & 1 \end{bmatrix} \leftarrow \begin{array}{l} R_2 \rightarrow R_2 - 1.5R_1 \\ R_3 \rightarrow R_3 - 0.5R_1 \end{array} \rightarrow \begin{bmatrix} 4 & 3 & 2 \\ 0 & 0.5 & 1 \\ 2 & 1 & 3 \end{bmatrix}$$

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 1.5 & 1 & 0 \\ 0.5 & 0 & 1 \end{bmatrix} \quad U = \begin{bmatrix} 4 & 3 & 2 \\ 0 & 0.5 & 1 \\ 0 & -0.5 & 2 \end{bmatrix}$$

$$R_3 \rightarrow R_3 + R_2$$

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 1.5 & 1 & 0 \\ 0.5 & -1 & 1 \end{bmatrix} \quad U = \begin{bmatrix} 4 & 3 & 2 \\ 0 & 0.5 & 1 \\ 0 & 0 & 3 \end{bmatrix}$$

Now multiplying $L \times U$, we get

$$= \begin{bmatrix} 4+0+0 & 3+0+0 & 2+0+0 \\ 6+0+0 & 4.5+0.5+0 & 3+1+0 \\ 2+0+0 & 1.5-0.5+0 & 1-1+1 \end{bmatrix}$$

$$= \begin{bmatrix} 4 & 3 & 2 \\ 6 & 5 & 4 \\ 2 & 1 & 3 \end{bmatrix}, \text{ which is } A$$

so $A = LU$

b) Cholesky Decomposition.

$A \rightarrow$ matrix of dim $n \times n$

$$A = LL^T$$

Let $A = \begin{bmatrix} 4 & 2 & 1 \\ 2 & 5 & 3 \\ 1 & 3 & 6 \end{bmatrix}$

$$A = \begin{bmatrix} 4 & 2 & 1 \\ 2 & 5 & 3 \\ 1 & 3 & 6 \end{bmatrix} = \begin{bmatrix} a & 0 & 0 \\ b & c & 0 \\ d & e & f \end{bmatrix} \begin{bmatrix} a & b & d \\ 0 & c & e \\ 0 & 0 & f \end{bmatrix}$$

$$= \begin{bmatrix} a^2 & ab & ad \\ ab & b^2+c^2 & bd+ce \\ ad & bd+ce & d^2+e^2+f^2 \end{bmatrix}$$

$$a = 2$$

$$b = 1$$

$$d = 0.5$$

$$c = 2$$

$$e = 1.25$$

$$f = 2.046$$

we get $L = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 2 & 0 \\ 0.5 & 1.25 & 2.046 \end{bmatrix}$

$$L L^T = \begin{bmatrix} 4+0+0 & 2+0+0 & 1+1.25+0 \\ 2+0+0 & 1+4+0 & 0.5+1.5+0 \\ 1+0+0 & 0.5+1.5+0 & 0.25+1.5625+4.19 \end{bmatrix}$$

$$= \begin{bmatrix} 4 & 2 & 1 \\ 2 & 5 & 3 \\ 1 & 3 & 6 \end{bmatrix} = A$$

$$\therefore A = L L^T$$

c) QR Decomposition

$A = QR$ where Q is orthonormal
 R is upper triangular

$$Q Q^T = I$$

$$A = \begin{bmatrix} 1 & 2 & 4 \\ 0 & 0 & 5 \\ 0 & 3 & 6 \end{bmatrix}$$

$$q_1' = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

$$r_{11} = \|q_1'\| = \sqrt{1} = 1$$

$$q_1 = \frac{1}{\|q_1'\|} \cdot q_1' = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

$$r_{12} = q_1^T \cdot a_2 = [1 \ 0 \ 0] \cdot \begin{bmatrix} 2 \\ 0 \\ 3 \end{bmatrix} = 2$$

$$q_2' = a_2 - r_{12} \cdot q_1 = \begin{bmatrix} 2 \\ 0 \\ 3 \end{bmatrix} - 2 \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 3 \end{bmatrix}$$

$$r_{22} = \|q_2'\| = \sqrt{3^2} = 3$$

$$q_2 = \frac{1}{\|q_2'\|} \cdot q_2' = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$$r_{13} = q_1^T a_3 = [1 \ 0 \ 0] \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} = 4$$

$$r_{23} = q_2^T a_3 = [0 \ 0 \ 1] \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} = 6$$

$$q_3' = a_3' - r_{13} q_1' - r_{23} q_2'$$

$$= \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} - 4 \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} - 6 \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 \\ 5 \\ 0 \end{bmatrix}$$

$$r_{33} = \|q_3'\| = 5$$

$$q_3 = \frac{1}{\|q_3'\|} q_3' = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

$$Q = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

$$R = \begin{bmatrix} 1 & 2 & 4 \\ 0 & 3 & 6 \\ 0 & 0 & 5 \end{bmatrix}$$

$$Q \times R = \begin{bmatrix} 1+0+0 & 2+0+0 & 4+0+0 \\ 0+0+0 & 0+0+0 & 0+0+5 \\ 0+0+0 & 0+3+0 & 0+6+0 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 2 & 4 \\ 0 & 0 & 5 \\ 0 & 3 & 6 \end{bmatrix} = A$$

d) Take a random 7 x 5 matrix having all its columns as linearly independent and its decompose into Q and R. What is your observation on the diagonal elements of R?

```
def run_q2d(verbose=True):  
  
    np.random.seed(50)  
  
    A = np.random.randint(1, 10, (7, 5)).astype(float)  
  
    print("\nRandom 7x5 Matrix:")  
    print(A)  
  
    Q, R = qr_decomposition(A, verbose)  
  
    print("\nQ:")  
    print(np.round(Q, 3))  
  
    print("\nR:")  
    print(np.round(R, 3))  
  
    print("\nDiagonal Entries of R:")  
  
    for i in range(R.shape[0]):  
        print(np.round(R[i][i], 3))  
  
    print("\nVerification:")  
    print(np.allclose(A, np.dot(Q, R)))
```

Inferences from the diagonal elements of R:

- 1) All positive values indicate linearly independent columns**
- 2) $|\det(A)| = \text{product of diagonal entries of R}$ (For matrix of dim. $N \times N$)**
- 3) The diagonal entries $R[i][i]$ represent the norms (lengths) of the orthogonalized column vectors**


```
(bits-pilani) Gopi ~/Documents/BITS-Pilani/MFML/Assignment 1/Code (main) $ python main.py
```

```
===== Q2(d) =====
```

```
Random 7x5 Matrix:
```

```
[[1. 1. 2. 5. 7.]  
 [6. 7. 7. 6. 3.]  
 [8. 5. 4. 7. 5.]  
 [2. 6. 1. 7. 4.]  
 [3. 4. 4. 4. 3.]  
 [1. 4. 3. 1. 4.]  
 [1. 1. 8. 4. 9.]]
```

```
=== QR Decomposition (Gram-Schmidt Process) ===
```

```
Initial Matrix A:
```

```
[[1. 1. 2. 5. 7.]  
 [6. 7. 7. 6. 3.]  
 [8. 5. 4. 7. 5.]  
 [2. 6. 1. 7. 4.]  
 [3. 4. 4. 4. 3.]  
 [1. 4. 3. 1. 4.]  
 [1. 1. 8. 4. 9.]]
```

```
Matrix dimensions: 7 x 5
```

```
Initial Q (zeros):
```

```
[[0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]]
```

```
Initial R (zeros):
```

```
[[0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]]
```

```
--- Processing column 0 ---
```

```
Starting with v = A[:, 0]:
```

```
[1. 6. 8. 2. 3. 1. 1.]
```

```
Normalizing:
```

```
R[0][0] = ||v|| = sqrt(v · v) = 10.770330  
Q[:, 0] = v / R[0][0]
```

```
Q[:, 0] (normalized):
```

```
[0.093 0.557 0.743 0.186 0.279 0.093 0.093]
```

=== Final QR Decomposition ===

Q (Orthonormal columns):

```
[ [ 0.093  0.006  0.139  0.655  0.289]
  [ 0.557  0.202  0.156 -0.319 -0.478]
  [ 0.743 -0.455 -0.223  0.086  0.329]
  [ 0.186  0.679 -0.311  0.482 -0.138]
  [ 0.279  0.184  0.114 -0.052 -0.172]
  [ 0.093  0.507  0.102 -0.432  0.729]
  [ 0.093  0.006  0.887  0.201  0.011]]
```

R (Upper triangular):

```
[ [10.77  10.399  9.378 11.885  8.821]
  [ 0.      5.988  2.585  4.077  3.72 ]
  [ 0.      0.      8.023  1.996  7.811]
  [ 0.      0.      0.      5.491  5.904]
  [ 0.      0.      0.      0.      4.183]]
```

Verification: $Q * R$ should equal A

Q:

```
[ [ 0.093  0.006  0.139  0.655  0.289]
  [ 0.557  0.202  0.156 -0.319 -0.478]
  [ 0.743 -0.455 -0.223  0.086  0.329]
  [ 0.186  0.679 -0.311  0.482 -0.138]
  [ 0.279  0.184  0.114 -0.052 -0.172]
  [ 0.093  0.507  0.102 -0.432  0.729]
  [ 0.093  0.006  0.887  0.201  0.011]]
```

R:

```
[ [10.77  10.399  9.378 11.885  8.821]
  [ 0.      5.988  2.585  4.077  3.72 ]
  [ 0.      0.      8.023  1.996  7.811]
  [ 0.      0.      0.      5.491  5.904]
  [ 0.      0.      0.      0.      4.183]]
```

Diagonal Entries of R:

10.77

5.988

8.023

5.491

4.183